PROJECT GUIDE

eGenome App Test Automation — How It Works

Architecture, tools and the six layers of the project · 30 Oct 2026 · Nishant

The big picture

The project is a robot tester for the eGenome Android app. A script on your PC drives the real app on an emulator or phone, the way a person would: it taps, types and reads the screen. After each action it checks that the right things happened, and it records a screenshot so a person can review the run later.

It is built in six layers. Each one uses the layer before it:

1. **Locator capture.** A tool that walks the app and saves how to find every button and label ("locators").
2. **Page objects.** One JavaScript class per screen, wrapping those locators in actions like `LoginPage.requestOtp(phone)`.
3. **Tests.** Specs written in plain steps ("log in, open Reports, open one report"), grouped into smoke, regression, layout and extended suites.
4. **Layout checks.** A comparison engine that checks text, position, size and alignment of every element against a saved baseline.
5. **Dashboard.** Every step, assertion and screenshot is recorded and turned into an HTML report.
6. **Deliverables.** A hand-over folder for testers: locator catalogue, test-case list, how-to guides and a dashboard snapshot.

The key idea to carry forward: tests never touch raw locators. They talk to page objects, and page objects talk to the app. When the app changes, you fix one page object instead of fifty tests.

The tools

Every tool here is free and open source except Android Studio's emulator images, which are free from Google. Versions are the ones installed in your project today.

Tool	What it is	Its job in this project
Node.js (v24)	JavaScript runtime for your PC	Runs every script: tests, capture tool, dashboard builder
npm + `package.json`	Node's package manager	Installs libraries; `npm run test:smoke` etc. are shortcuts defined in `package.json`
Android SDK / Android Studio	Google's Android tools	Provides the emulator and `adb`
adb (Android Debug Bridge)	Command-line link to a device	Lists devices, installs apps, reads logs; Appium uses it under the hood
Appium (3.9)	Automation server	Receives commands like "tap this" over HTTP and carries them out on the device
UiAutomator2 driver (8.7)	Appium's Android plug-in	Installs a helper app on the phone that can see and touch any screen element
WebdriverIO (10.1)	Test framework for JavaScript	Talks to Appium; gives you `$()`, `click()`, `getText()`, waits, and runs the suites
Mocha	Test runner style	`describe` / `it` blocks, hooks such as `before` and `after`
expect-webdriverio	Assertion library	`await expect(button).toBeDisplayed()`
dotenv	Config loader	Reads `.env` (phone number, OTP, device ID) so secrets stay out of code

Tool	What it is	Its job in this project
@xmldom/xmldom + xpath	XML parsing	Read the screen's element tree for layout checks
Plain HTML + JavaScript	No framework	The dashboard: one `index.html` that loads data files, works offline

Two ideas make this stack click:

- **WebDriver is a protocol.** WebdriverIO sends standard HTTP requests ("find element", "click") to Appium. That is why the same skills transfer to web testing with Selenium or Playwright.
- **The app is React Native.** Most elements have no IDs, so we find them by their accessibility label (`content-desc`) or visible text. That decision shapes the whole locator strategy.

How a test run flows

One command sets off a chain: your PC drives the phone through Appium, and every step is recorded on the way back.

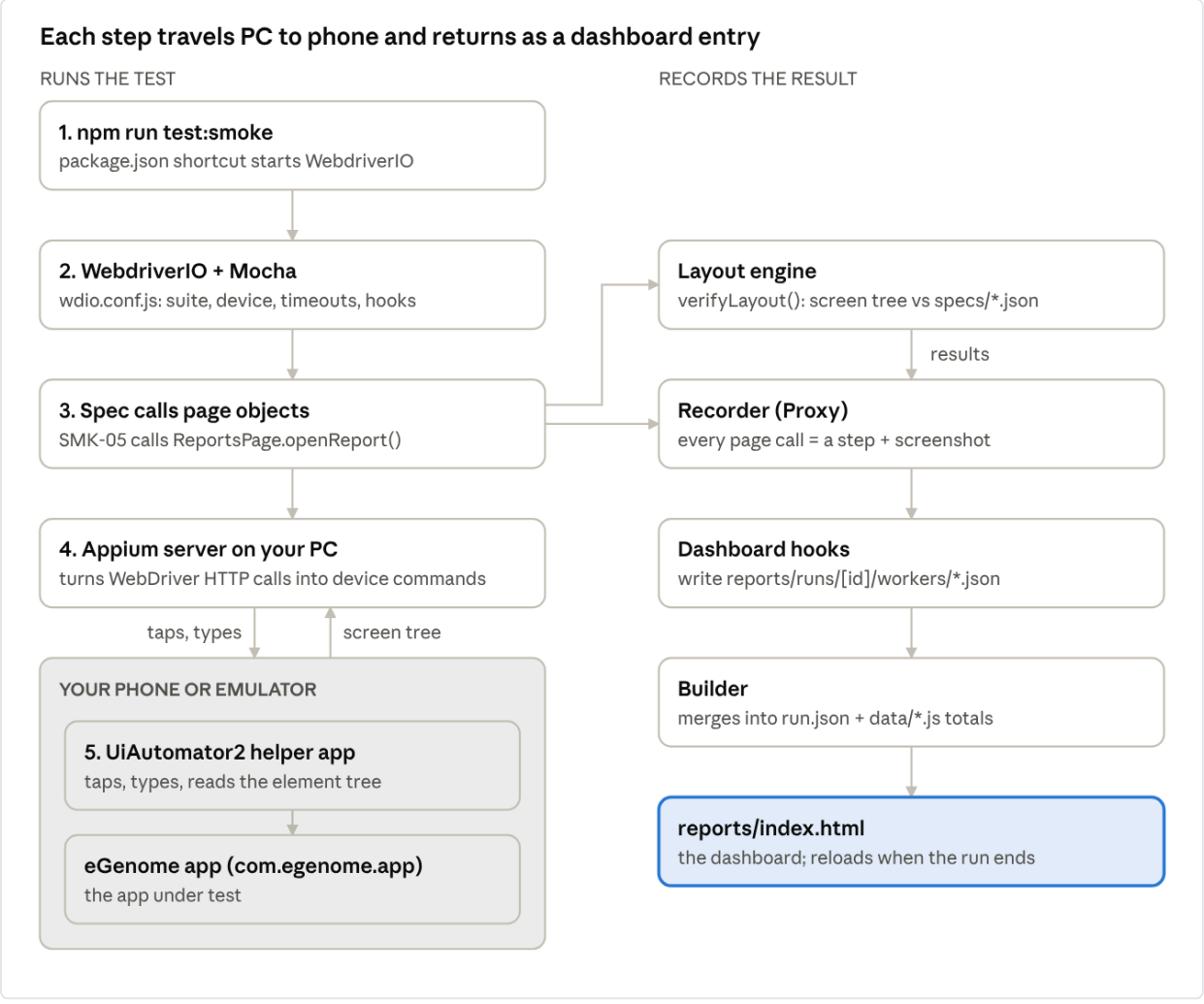


Figure 1 · One test run: PC, phone and report

Left column: commands go down to the phone, and the phone sends the screen back up. Right column: each page-object call is recorded as a step, checked against layout specs where asked, and built into the dashboard when the run ends.

Where everything lives

Each top-level folder is one layer. Learn this map first; every later section points into it.

```
eGenome.ai/
├── wdio.conf.js            WebdriverIO settings: device, app, suites, timeouts, hooks
├── package.json            libraries + npm run shortcuts
├── .env / .env.example     your phone number, OTP, device ID (never committed)
├── locators/               Layer 1 output: one JSON per captured screen
├── scripts/
│   ├── locator-capture/    Layer 1: the capture tool
│   ├── layout/             generate / baseline / accept layout specs
│   └── deliverables/       Layer 6: builds the tester hand-over folder
├── test/
│   ├── pageobjects/        Layer 2: BasePage + one class per screen
│   ├── data/users.js       test users, account data, feature flags
│   ├── specs/              Layer 3: smoke/ regression/ layout/ extended/
│   ├── layout/specs/       Layer 4: one baseline JSON per screen
│   └── support/
│       ├── app.js          restart app, clear data, toggle network
│       ├── layout/         Layer 4: the comparison engine
│       └── dashboard/      Layer 5: recorder, hooks, builder, HTML template
├── reports/                Layer 5 output: runs/, data/, index.html
└── deliverables/           Layer 6 output
```

Layer 1 — Capturing locators

A locator is the address of an element on screen. Before you can automate a screen you need one for every button, field and label on it. The capture tool collects them automatically while you click through the app.

How it works:

1. It opens an Appium session and asks for the **page source**: an XML tree of every element on screen, with attributes such as `text`, `content-desc`, `resource-id`, `class`, `clickable` and `bounds` (its rectangle in pixels).
2. It works out which screen you're on (from the current activity plus the visible titles) and keeps only useful elements: ones with text, a label, or that can be tapped or typed into.
3. For each element it builds a **primary selector** plus fallbacks, preferring the most stable attribute: resource-id, then accessibility label, then text, then XPath as a last resort. It also checks each selector matches exactly one element.
4. It saves `locators/<Screen>.json`, and in watch mode it repeats every second or two, so new screens are captured as you navigate.

What to learn from it: **the page source is the ground truth.** Whenever a test can't find something, dump the page source and look at what Android actually reports. The address line-break bug was found exactly this way.

Layer 2 — Page objects

A page object is a class that knows one screen: how to find its elements and how to do things on it. Tests call `await ReportsPage.openReport('CVD Assessment')`; only `ReportsPage.js` knows which selector that uses.

The pieces:

- **`BasePage.js`** — the parent class. It has the shared actions every screen needs: `tap`, `type`, `scrollTo`, `waitForOpen`, `isOpen`, `goBack`, `hideKeyboard`, `verifyLayout`.

- **Selector helpers (`S`)** — small functions that build UiAutomator selectors: `S.desc('Continue')`, `S.textMatches('[0-9]+ reports? found')`, `S.descContains(', Manage Addresses')`. Partial matching survives live data and icon characters in labels.
- **`anchors`** — each page lists one or two elements that prove the screen is open. `waitForOpen()` polls until one is visible.
- **One class per screen** — `LoginPage`, `OtpPage`, `HomePage`, `ReportsPage`, `SettingsPage` and so on. Shared parts such as the bottom tab bar are their own component (`BottomNav`).
- **`index.js`** — exports every page wrapped in a recorder, so each method call automatically becomes a dashboard step (Layer 5).

A trimmed example of the pattern:

```
const SEL = {
  title: S.text('OTP, please'),
  digit: (n) => S.desc(`OTP digit ${n}`),
  continueBtn: S.desc('Continue'),
};

class OtpPage extends BasePage {
  get anchors() { return [SEL.title]; }

  async enterOtp(code) {
    for (let i = 0; i < 6; i++) await $(SEL.digit(i + 1)).setValue(code[i]);
  }

  async submitOtp(code) {
    await this.waitForOpen();
    await this.enterOtp(code);
    await this.tap(SEL.continueBtn);
  }
}
export default new OtpPage();
```

Two habits worth copying: **wait for state, never sleep** (`waitForDisplayed`, `waitUntil`), and **give every failure a readable message** ("OTP digit 3 not displayed") so the dashboard tells the tester what went wrong.

Layer 3 — Test data, OTP login and the suites

Test data stays out of the code. `.env` holds the device ID, test phone number, fixed OTP and account names. `test/data/users.js` reads it into objects (`getUser()`, `accountData`, `flags`), so the same tests run against any account or device.

OTP login works because the backend cooperates. Real SMS can't be automated reliably, so staging has a test number with a fixed OTP. The tests type it into the six boxes like a person would. This is the standard answer for OTP, magic-link and 2FA apps: ask developers for a test-only bypass on staging, never in production.

A spec is a list of user stories. Each `it(...)` is one test case with an ID and priority, written in page-object calls:

```
it('SMK-05 [P0] Report tab lists reports and one report opens', async () => {
  await ReportsPage.open();
  await ReportsPage.verifyLayout();
  expect(await ReportsPage.getReportCount()).toBeGreaterThan(0);
  await ReportsPage.openReport(accountData.completedReport);
  await ReportDetailPage.waitForOpen();
});
```

The suites are chosen by how often they run and how deep they go:

Suite	Tests	When to run	What it covers
Smoke	8 (SMK-01…08)	Every build, about 5 min	Login, Home, Reports, one report, Settings, every screen once, sign out
Regression	60 (LGN, HOM, RPT, PAY, SET, RES)	Nightly or before release	Every P0/P1 flow: validation, OTP edge cases, data round trips, network loss
Layout	16 (LAY-01…16)	When UI changes	One layout check per screen; also used to re-record baselines
Extended	25 pending	When prerequisites exist	P2 and destructive tests, each marked with what it still needs

Three design rules keep the suites reliable:

- **Start from a known state.** Smoke and login clear the app's data first; other specs call `ensureLoggedIn()`.
- **Leave no trace.** Tests that change data put it back (weight +1 kg, then restored). Ones that can't are switched off by default (`TEST_ALLOW_DATA_CHANGES`).
- **Skip, don't fail, when data is missing.** No pending payment on the account means the payment test skips with a reason, not a red failure.

Layer 4 — Layout verification

The layout engine answers "does this screen still look right?" without comparing pixels. Pixel comparison breaks on every font tweak, animation or device. Instead it compares the **element tree** against a saved spec, in proportions of the screen.

It has three parts, all in `test/support/layout/`:

1. **Reader (`hierarchy.js`).** Parses the page source into a flat list of elements with boxes, and evaluates selectors offline (the same strings the page objects use).
2. **Spec builder (`spec-builder.js`).** Turns one screen into a JSON baseline. For each element it records how to find it, its text, its box as fractions (x 6.7%, y 71.3%, width 86.7%), and which checks apply. Then it derives rules between elements: inside, same row, top-to-bottom order, gaps, left/right alignment, centring, no overlap.
3. **Checker (`check.js`).** A pure function: spec + current screen in, a list of pass/fail/warning results out. No device needed, so it can be unit-tested.

The tricky part is **things that change on their own**. These are the rules that make it stable:

Element kind	How it's matched	What's checked
Fixed text and labels	Exactly, by label or text	Text, position, size, state
Live values (prices, dates, counts)	By format, e.g. `₹[0-9][0-9,.]*`	Format and position, not the value
Account names	`{{TEST_USER_NAME}}` tokens filled from `.env`	Exact text for the configured account
Repeated or unlabelled items (bars, icons, images)	Closest match to the expected box	Presence, position, height

Element kind	How it's matched	What's checked
Calendar days	One grid rule for the whole month	Every day once, under the right weekday, even rows

Positions are only compared when the screen has the same shape (aspect ratio) as the baseline; on a taller phone, relations are still checked. Tap targets under 48 dp are reported as warnings, never failures.

The workflow: `npm run layout:baseline` records specs from the live app. Each run saves the screen tree beside its results. `npm run layout:accept` rebuilds a spec from a run when a difference turns out to be intended. Specs stay `"reviewed": false` until someone compares them with the design.

Layer 5 — Dashboard and reporting

The dashboard shows every run, test and step, with a screenshot after each step. It needs no server: it's one HTML file that loads JavaScript data files, so it opens straight from disk.

How data gets from a test into the page:

1. **Recording (`recorder.js`).** Page objects are wrapped in a JavaScript `Proxy`. Every method call becomes a *step*: title, start/end time, pass/fail, error, a screenshot, and any extras such as layout results or warnings. Assertions are recorded through WebdriverIO's `afterAssertion` hook.
2. **Hooks (`hooks.js`).** WebdriverIO calls hooks at fixed moments: `onPrepare` (run starts), `beforeTest`, `afterTest`, `onComplete` (run ends). The hooks start and end each test record, collect the device log (logcat) on failure, and write `reports/runs/<id>/workers/<worker>.json`. `withDashboard(config)` in `wdio.conf.js` plugs them all in.
3. **Building (`builder.js`).** Merges the worker files into `run.json`, computes totals (tests, steps, layout checks, warnings) and writes `reports/data/index.js` (all runs) and `runs/<id>.js`. Old runs past `DASHBOARD_KEEP_RUNS` are deleted.
4. **Viewing (`template/index.html`).** Plain JavaScript renders an overview (headline cards, consolidated totals, trend chart, runs table, per-test history) and a run page (summary boxes, filter tabs, step tables, screenshot viewer with layout boxes drawn on top). It polls a tiny `latest.js` every 5 seconds and reloads when a run finishes.

Why data as `.js` and not `.json`? Browsers block `fetch()` from `file://` pages, but they do load `<script>` files. Writing `window.DASHBOARD_INDEX = {...}` sidesteps that without a server. It's a useful trick for any offline report.

Layer 6 — Deliverables for testers

`npm run deliverables` builds a `deliverables/` folder a manual tester can use without reading code. It is generated from the project itself, so it never goes stale:

- **Locator catalogue** — every screen, its page object and selectors, read from `locators/` and each page's `LOCATORS` export.
- **Test-case inventory** — every `it(...)` across the suites with its ID, priority, suite, and any `[needs: …]` prerequisite, found by reading the spec files.
- **Layout specs** — a readable summary of each screen's baseline for design review.
- **Guides** — setup, running tests, test data, known issues and recommendations for developers (Markdown in `scripts/deliverables/docs/`).
- **Dashboard snapshot** — the last few runs with screenshots, viewable offline.

The lesson: **generate documentation from the source of truth.** A test list typed by hand is wrong within a week; one parsed from the specs is always right.

Lessons from the problems we hit

Every bug below came up during this project. Each one teaches a habit worth keeping.

Symptom	Real cause	Fix	Habit to keep
Smoke "stuck" on the OTP screen	The layout check failed first, so the OTP was never typed	Re-baselined from the real screen state	Read the first error, not the last screenshot
One failure failed every later test	The Premium screen was left open	Tab navigation presses Back until the tab bar shows	Each test recovers to a known state
Commands took 40–90 s on Addresses	UiAutomator2 waits for an idle UI; an animation never stops	`waitForIdleTimeout: 100`	When everything is slow, suspect a global wait
"Could not scroll to 2 reports found"	`\d` in selectors was double-escaped on its way to the device	Selectors rewritten without backslashes (`[0-9]`)	Test the selector, not just the logic around it
Address text never matched	The text contained a line break	Patterns accept any whitespace	Look at the raw page source
Phone install timed out at 20 s	The phone waited for an "Install?" prompt	Longer install timeouts, OEM developer settings	Real devices need human steps once
Old run stuck as "Running"	Run stopped before `onComplete`	Marked aborted after 30 min	Design reports for interrupted runs

The pattern behind most of these: **baselines and selectors were built from captures, not from the live screen.** Whenever possible, record expectations from the real app in the exact state the test sees.

Your learning path

Build it again in this order, one milestone at a time. Each step should run before you start the next.

1. **JavaScript basics for automation.** Variables, functions, objects, arrays, classes, `import` / `export` , and above all `async` / `await` . Every device command is asynchronous; forgetting `await` is the most common bug.
2. **Set up the device side.** Install Android Studio, create an emulator, run `adb devices` , install your app, and turn on USB debugging on a phone.
3. **First Appium session.** Install Appium and the UiAutomator2 driver, start the server, and from a tiny script open the app and print `driver.getPageSource()` . Read that XML until it makes sense.
4. **Explore with Appium Inspector.** It shows the same tree visually, with each element's attributes. Practice writing a selector for any element by hand.
5. **First WebdriverIO test.** `npm init wdio` sets up `wdio.conf.js` . Write one `it(...)` that opens the app and checks the login screen shows.
6. **Introduce page objects.** Move selectors into a `LoginPage` class, then a `BasePage` with shared `tap` / `waitForOpen` . Rewrite the test to use only page methods.
7. **Config and test data.** Add `dotenv` and a `users.js` . Nothing environment-specific stays in code.
8. **Grow the suites.** Add a smoke suite (happy path across the app), then regression (validation, edge cases), with IDs and priorities in test names.
9. **Reporting.** Start with a built-in reporter (`spec` , then Allure). Later, write your own hooks to collect steps and screenshots, as this project's dashboard does.
10. **Stabilise.** Recover to a known state between tests, tune timeouts, add retries only for real flakiness, and run on a real phone.
11. **Advanced checks.** Layout verification, API calls to set up data, accessibility checks, performance timings.
12. **Automate the runs.** Run the suite in CI (GitHub Actions, Jenkins) on every build, with an emulator or a device cloud such as BrowserStack.

To practise, take any app and write the capture → page objects → smoke test chain for its login flow. Use this project as the answer key when you get stuck: every file is short enough to read in one sitting.